ARTIFICIAL INTELLIGENCE

TOPIC 3

Shallow networks and deep learning fundamentals

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Explain what a network represents; calculate shapes and parameter counts; choose a loss; derive backpropagation through a small network; understand optimization, initialization and regularization; diagnose training and generalization.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

1 Deep learning learns the representation too **4**

2 A shallow network is a flexible family of functions **4**

2.1 One input, hidden units and one output 4

2.2 Worked piecewise-linear function 4

2.3 Why activation functions matter 5

3 Multiple inputs and outputs: shapes before arithmetic **5**

3.1 Capacity and universal approximation 5

4 Deep networks compose representations **6**

5 Scores, probabilities and task-specific losses **6**

5.1 Classification scores 6

5.2 Multiclass hinge loss 6

5.3 Worked loss comparison 7

5.4 Regression and a regularized dataset objective 7

6 Gradient descent, momentum and adaptive optimizers **7**

6.1 Loss gradients drive learning 7

6.2 Momentum 7

6.3 Adam 8

7 Backpropagation: local derivatives and the chain rule **8**

7.1 Forward pass, backward pass, update 8

7.2 A complete worked scalar network 9

8 Matrix backpropagation and automatic differentiation **9**

8.1 One affine layer 9

8.2 Softmax with cross-entropy 9

8.3 A framework-independent training loop 10

9 Initialization and normalization **10**

9.1 Why hidden weights cannot all start identically 10

9.2 Input normalization and batch normalization 10

10 Regularization: more than a penalty term **10**

10.1 Explicit and implicit mechanisms 11

10.2 Dropout and its scaling 11

10.3 Data augmentation and other data strategies 11

11 Training behavior, generalization and double descent **11**

11.1 Read training and validation together 11

11.2 The modern capacity picture 12

12 Revision questions and answers **12**

12.1 Questions 12

12.2 Answers 12

13 Source guide **13**

1 Deep learning learns the representation too

Teacher slides: 3.1. Introduction to Deep Learning, pp. 6–10, 27–40.

AI is the broad study of intelligent systems; machine learning learns from experience; deep learning uses layered models to learn increasingly useful representations. Deep does not mean a model understands the task in a human sense. Big data describes a data-processing challenge rather than a particular learning algorithm.

A conventional pipeline can use manually designed features followed by a classifier. A deep network can learn the feature extractor and predictor together, using a loss tied to the final task. This changes what is trainable, but preserves the need to define a task, select data, avoid leakage, validate and analyze failures.

The introduction slides illustrate recognition, language, control, healthcare, science and recommendation tasks. Treat these as examples of applications rather than evidence that a model trained for one domain automatically transfers to another. The recurring requirement is useful data coverage and appropriate labels: class labels for classification, coordinates for localization, or pixel labels for segmentation.

2 A shallow network is a flexible family of functions

Teacher slides: 3.2. Shallow NN, pp. 2–22.

2.1 One input, hidden units and one output

For D hidden units, use

$$h_j(x) = a(b_j + w_j x), \qquad f(x) = b_o + \sum_{j=1}^{D} v_j h_j(x).$$

The activation a acts separately on each hidden preactivation. For ReLU,

$$\mathrm{ReLU}(z) = \max(0, z).$$

Each hidden unit first draws an affine function, then removes its negative portion. Its output weight changes its contribution to the final sum. With three units there are 3 input weights, 3 hidden biases, 3 output weights and 1 output bias: 10 trainable parameters, as in the slide example.

2.2 Worked piecewise-linear function

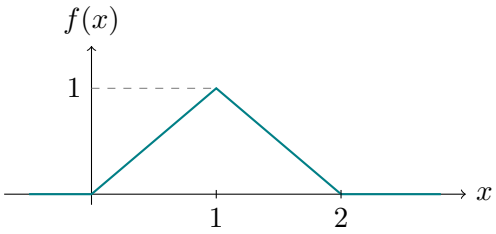
Choose

$$f(x) = \mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2).$$

The output is

$$f(x) = \begin{cases} 0, & x \le 0, \\ x, & 0 < x \le 1, \\ 2 - x, & 1 < x \le 2, \\ 0, & x > 2. \end{cases}$$

Three simple hinges create a triangular shape. The breakpoints arise where hidden preactivations cross zero. The combination is piecewise linear, even though each layer contains nonlinear operations.



2.3 Why activation functions matter

Without a nonlinear activation, stacking affine transformations produces only another affine transformation:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

More linear layers alone cannot create this triangular function. Sigmoid and tanh are smooth but saturate; ReLU is simple and has a constant positive-side derivative, but a unit can stop learning if its preactivation stays negative. The activation is usually a design choice, not an ordinary learned weight.

3 Multiple inputs and outputs: shapes before arithmetic

Teacher slides: 3.2. Shallow NN, pp. 26–42, 49–51.

For input dimension d, hidden width D and output dimension m,

$$h = a(W_1x + b_1), \qquad f = W_2h + b_2,$$

where

$$W_1 \in \mathbb{R}^{D\times d}, \quad b_1 \in \mathbb{R}^D, \quad W_2 \in \mathbb{R}^{m\times D}, \quad b_2 \in \mathbb{R}^m.$$

The parameter count is $D(d + 1) + m(D + 1)$. The slide example $d = 3$, $D = 3$, $m = 2$ therefore has $12 + 8 = 20$ parameters.

Different outputs share the same hidden representation but combine it with different output weights. For a two-dimensional input, each hidden unit has a straight activation boundary; its active side contributes an affine surface. Summing active contributions makes a piecewise-affine function over polygonal regions.

3.1 Capacity and universal approximation

Teacher slides: 3.2. Shallow NN, pp. 23–25, 43–47.

The universal approximation result in the slides says that, with enough hidden units and an appropriate activation, a shallow network can approximate a continuous function on a compact domain to arbitrary accuracy. It is an existence result. It does not say a finite dataset identifies that function, training finds the required parameters, or predictions generalize outside the domain.

With D hyperplanes in a d-dimensional input space, the maximum number of activation regions in general position is

$$\sum_{j=0}^{\min(d,D)} \binom{D}{j}.$$

For $d = 1, D = 3$, the maximum is 4 intervals; for $d = 2, D = 3$, it is $1 + 3 + 3 = 7$ regions. Coincident or otherwise dependent boundaries create fewer regions. Output weights can also make neighboring regions share the same affine output. The regions live in *input dimension* d, not output dimension or hidden width.

4 Deep networks compose representations

Teacher slides: 3.3. Deep Learning Fundamentals, pp. 3–14.

Let $h^{(0)} = x$. For hidden layers $\ell = 1, \ldots, L$,

$$z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}, \quad h^{(\ell)} = a^{(\ell)}(z^{(\ell)}).$$

An output layer maps the last hidden representation to task-specific scores or predictions. If widths are $d_0, \ldots, d_{L+1}$, including input and output,

$$\text{parameter count} = \sum_{\ell=1}^{L+1} d_\ell (d_{\ell-1} + 1).$$

For $4 \to 5 \to 3 \to 2$, this is $5(5) + 3(6) + 2(4) = 51$. State your depth convention: some descriptions count hidden layers, others count all layers with weights.

Composition allows later units to combine features detected earlier. This can express some functions much more efficiently than a single very wide hidden layer. The slides motivate this with increased numbers of linear regions and hierarchical structure. Efficiency is function-dependent; depth alone does not guarantee useful generalization or easy optimization.

5 Scores, probabilities and task-specific losses

Teacher slides: 3.3. Deep Learning Fundamentals, pp. 16–28.

5.1 Classification scores

A classifier produces a score vector $z = f_\theta(x)$. Raw scores, also called logits, need not sum to one and can be negative. For a single correct class, softmax turns logits into probabilities and cross-entropy assesses the labeled class:

$$p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}, \qquad \ell = -\log p_y = -z_y + \log \sum_j e^{z_j}.$$

For numerical stability, evaluate the log-sum-exp after subtracting the maximum logit. A probability of one for the correct class gives limiting loss zero; a probability tending to zero gives unbounded loss. Equal logits over C classes give $\log C$, a useful initialization check.

5.2 Multiclass hinge loss

An alternative shown in the slides is

$$\ell_{\text{hinge}} = \sum_{j \neq y} \max(0, z_j - z_y + \Delta).$$

It demands a score margin Δ, rather than interpreting outputs as normalized probabilities. Once all margins are met, further score separation need not reduce hinge loss, whereas softmax loss continues to respond to class probability.

5.3 Worked loss comparison

Suppose logits are $(2, 1, 0)$ and the first class is correct. Softmax probability is

$$p_1 = \frac{1}{1 + e^{-1} + e^{-2}} \approx 0.665, \quad \ell_{\mathrm{CE}} \approx 0.408.$$

With $\Delta = 1$, hinge loss is $\max(0, 1 - 2 + 1) + \max(0, 0 - 2 + 1) = 0$. The hinge margin is met, but the probabilistic loss still recognizes uncertainty.

5.4 Regression and a regularized dataset objective

For a continuous target, use an unconstrained output with an appropriate regression loss, for example $\frac{1}{2}\|\hat{y} - y\|^2$. A multiclass softmax would impose the wrong output constraint for arbitrary continuous regression. In general,

$$J(\theta) = \frac{1}{n} \sum_i \ell(f_\theta(x_i), y_i) + \lambda \mathcal{R}(\theta).$$

The data term measures fit; the regularizer expresses preferences for parameters or functions. Parameters, model capacity, optimizer settings and preprocessing must all be tuned against validation behavior rather than only training loss.

6 Gradient descent, momentum and adaptive optimizers

Teacher slides: 3.3. Deep Learning Fundamentals, pp. 29–42.

6.1 Loss gradients drive learning

Gradient descent uses $\theta \leftarrow \theta - \eta \nabla J$. Mini-batch SGD replaces the full average gradient by a sampled average. The gradient is an instantaneous direction of steepest increase; the minus sign makes a minimizing step.

Neural-network objectives are generally nonconvex. A zero gradient is not enough to identify a minimum. At a critical point, a positive-definite Hessian indicates a strict local minimum, a negative-definite Hessian a strict local maximum, and mixed positive/negative eigenvalues a saddle. Zero Hessian eigenvalues leave the simple classification inconclusive.

Batch size affects gradient noise, compute and memory. An epoch is a pass through the training data; a step is one parameter update. With n samples and batch size B, there are roughly $\lceil n/B \rceil$ updates per epoch if the last batch is retained.

6.2 Momentum

One consistent convention is

$$v_t = \beta v_{t-1} + (1 - \beta) g_t, \qquad \theta_t = \theta_{t-1} - \eta v_t.$$

Momentum averages gradients over time, reducing oscillation and maintaining a useful direction through shallow regions. Some implementations omit $(1-\beta)$ and rescale the effective learning rate; do not combine equations from different conventions without checking.

6.3 Adam

Adam tracks first and second moments:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\ \hat{m}_t &= m_t/(1-\beta_1^t), \quad \hat{v}_t = v_t/(1-\beta_2^t), \\ \theta_t &= \theta_{t-1} - \eta \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon). \end{aligned}$$

Squares, roots and division are coordinatewise. Bias correction compensates for initializing moment estimates at zero. The denominator rescales each coordinate, while the numerator contains momentum. Adam is not guaranteed to outperform SGD or find a global optimum.

With ordinary SGD, an L2 penalty produces a weight shrinkage term. With adaptive coordinate scaling, inserting L2 into the gradient need not equal uniform weight decay. AdamW decouples decay from the adaptive gradient step, as the slides note. Learning-rate schedules can reduce step size as fitting progresses.

7 Backpropagation: local derivatives and the chain rule

Teacher slides: 3.4. Deep Learning Fundamentals II, pp. 3–22.

7.1 Forward pass, backward pass, update

Forward propagation evaluates operations and stores intermediates needed for differentiation. Backpropagation propagates derivatives of the scalar loss backward through those operations. The optimizer then uses the parameter gradients to update weights. Backpropagation is a differentiation algorithm, not itself the optimization algorithm.

For $u = g(x)$ and $L = f(u)$, the chain rule is $dL/dx = (dL/du)(du/dx)$. In a graph with several paths from a variable to the loss, their derivative contributions add.

Forward operation	Backward rule with upstream scalar derivative g
$z = x + y$	Send g to both inputs.
$z = xy$	Send gy to x, and gx to y.
$z = \mathrm{ReLU}(x)$	Send $gI(x > 0)$; a convention is needed at zero.
$z = \max(x, y)$	Route derivative to the winning input; handle ties by convention.

The practical implementation needs products of local derivatives with upstream derivatives, not a huge explicitly materialized Jacobian for the entire network.

7.2 A complete worked scalar network

Set $x = 2$, hidden weight $w = 1$, hidden bias $b = -1$, output weight $v = 3$ and output bias $c = 0$. Target is $y = 4$:

$$z = wx + b = 1, \quad h = \mathrm{ReLU}(z) = 1, \quad \hat{y} = vh + c = 3, \quad L = \tfrac{1}{2}(\hat{y} - y)^2 = 0.5.$$

Backward pass:

$$\frac{\partial L}{\partial \hat{y}} = -1, \qquad \frac{\partial L}{\partial v} = (-1)h = -1, \qquad \frac{\partial L}{\partial c} = -1,$$
$$\frac{\partial L}{\partial h} = (-1)v = -3, \qquad \frac{\partial L}{\partial z} = -3I(z > 0) = -3,$$
$$\frac{\partial L}{\partial w} = (-3)x = -6, \qquad \frac{\partial L}{\partial b} = -3.$$

With learning rate 0.01, update all parameters from these same old-parameter gradients:

$$w = 1.06, \quad b = -0.97, \quad v = 3.01, \quad c = 0.01.$$

New hidden activation is 1.15, prediction 3.4715, and loss is approximately 0.1397. The loss falls on this example. A much larger step need not do so.

Two easily confused derivatives

The derivative of the loss with respect to a hidden activation is an upstream error signal. The derivative with respect to a weight also multiplies the input that the weight acted on. For the example, $\partial L/\partial h = -3$, while $\partial L/\partial w = -6$. Keep their meanings and shapes distinct.

8 Matrix backpropagation and automatic differentiation

Teacher slides: 3.4. Deep Learning Fundamentals II, pp. 18–22, 44–53.

8.1 One affine layer

Using column vectors, $z = Wh + b$. Given upstream derivative $\delta = \partial L/\partial z$,

$$\frac{\partial L}{\partial W} = \delta h^T, \quad \frac{\partial L}{\partial b} = \delta, \quad \frac{\partial L}{\partial h} = W^T \delta.$$

The outer product has exactly the shape of W. Passing through ReLU multiplies the upstream activation derivative elementwise by $I(z > 0)$. In a batch, sum or average contributions consistently with the loss reduction.

For batch rows $H \in \mathbb{R}^{B \times d}$ and $Z = HW^T + \mathbf{1}b^T$, a derivative matrix $D = \partial L/\partial Z$ gives $\partial L/\partial W = D^T H$, $\partial L/\partial H = DW$, and bias derivative equal to the row sum of D. Do not apply column-vector formulas blindly to row-oriented code.

8.2 Softmax with cross-entropy

For one example and one-hot target y, the logit derivative simplifies to

$$\frac{\partial \ell}{\partial z} = p - y.$$

This is an output-layer error vector that backpropagates through the network. For mean loss, account for the division by batch size once; duplicating it artificially shrinks gradients.

8.3 A framework-independent training loop

```
for each mini-batch:
    clear accumulated parameter gradients
    predictions = model.forward(inputs)
    loss = task_loss(predictions, targets) + regularizer
    loss.backward()   # local derivatives through the graph
    optimizer.step()  # use gradients to change parameters
```

This pseudocode explains the process without depending on a particular library version. Automatic differentiation records differentiable operations, applies their local rules and accumulates gradients at shared inputs. It still requires a correctly defined forward model and loss.

Backpropagation is efficient but needs stored intermediates and reverse dependencies, which explains the slides' memory and sequential-computation limitations. Numerical finite differences can check a small derivative away from ReLU kinks; they are too expensive for routine large-model training.

9 Initialization and normalization

Teacher slides: 3.4. Deep Learning Fundamentals II, pp. 23–25.

9.1 Why hidden weights cannot all start identically

Identical hidden units can receive identical gradients and remain identical, wasting capacity. Random initialization breaks symmetry. Zero biases are often acceptable; zeroing all hidden weights is different.

The scale also matters. Across many layers, poorly scaled activations and derivatives can shrink toward zero or grow excessively. For a ReLU layer with d_{in} incoming units, He initialization uses a weight variance around $2/d_{\text{in}}$. Its rationale is to preserve useful signal variance despite ReLU zeroing part of the input. Different activations can require different initialization assumptions.

9.2 Input normalization and batch normalization

Normalize inputs using training-set statistics. Batch normalization instead normalizes intermediate quantities using current mini-batch statistics during training and learns an affine rescaling:

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma_{\text{BN}}\hat{z} + \beta_{\text{BN}}.$$

The two affine parameters are learned, usually per feature or channel. They are unrelated to the RL discount factor. In common batch-normalization use, inference employs accumulated statistics rather than the statistics of each individual test batch. Training and inference modes therefore matter.

10 Regularization: more than a penalty term

Teacher slides: 3.4. Deep Learning Fundamentals II, pp. 26–35.

10.1 Explicit and implicit mechanisms

An explicit L2 term $\lambda \sum_\ell \|W^{(\ell)}\|_F^2$ contributes gradient $2\lambda W^{(\ell)}$. It is local to those weights and does not need a downstream activation error to be computed. Biases are often left unpenalized.

Early stopping selects a checkpoint before validation performance deteriorates. Ensembling averages predictions from different trained models. SGD's optimization trajectory also creates implicit preferences, though it does not guarantee good generalization in every setting. These mechanisms should be evaluated using held-out data.

10.2 Dropout and its scaling

During training, independently retain a hidden activation with probability q and discard it otherwise. In *inverted dropout*,

$$\tilde{h}_j = \frac{m_j}{q} h_j, \qquad m_j \sim \text{Bernoulli}(q).$$

Then $\mathrm{E}[\tilde{h}_j \mid h_j] = h_j$. Ordinary deterministic inference uses the full network without dropout; training-time scaling has already compensated for the missing units. The older convention performs compensation at inference instead. Do not scale in both stages.

If $h = 6$ and $q = 0.5$, the training output is either 0 or 12, with expectation 6. Monte Carlo dropout keeps dropout active across multiple inference runs to obtain a variability estimate, as the slides mention; interpreting it as uncertainty depends on modeling assumptions.

10.3 Data augmentation and other data strategies

Augmentation creates variations that preserve the target semantics. Flipping a natural object image may be appropriate; flipping a digit or changing a physically meaningful orientation can change its label. Apply stochastic augmentation to training data, while using consistent evaluation transforms.

The slides also name transfer learning, multi-task learning and self-supervised approaches. Their shared purpose is to obtain useful structure beyond a small labeled dataset. Transfer learning is developed in the CNN notes; the others are introductory mentions rather than complete algorithms in the supplied decks.

11 Training behavior, generalization and double descent

Teacher slides: 3.4. Deep Learning Fundamentals II, pp. 36–42; 3.3. Deep Learning Fundamentals, pp. 43–44.

11.1 Read training and validation together

If both losses remain poor, suspect underfitting, an unsuitable learning rate, insufficient optimization or an implementation problem. If training improves but validation worsens, examine overfitting, split mismatch and regularization. A noisy training curve can reflect small batches; persistent spikes can also indicate unstable steps or data problems. A gap alone does not identify its cause.

Hyperparameters include layer widths and depth, activation choices, regularization, learning rate and its schedule, and batch size. Change them with a defined validation procedure. The teacher's

suggested experiment with random data is useful for checking execution, but fitting random labels does not demonstrate meaningful generalization.

11.2 The modern capacity picture

The classical bias–variance illustration suggests test error first falls and then rises as capacity increases. Double descent adds a regime where error may peak around an interpolation threshold and then fall again as capacity grows further. Optimization, data, model family and regularization affect this behavior.

Do not turn the slide example into a rule that bigger networks always generalize better. Many parameter configurations can interpolate training data; the learned function and the optimization path still matter. Capacity, fitted complexity and quality on unseen data are distinct concepts.

12 Revision questions and answers

12.1 Questions

1. How many parameters are in a $3 \to 4 \to 2$ fully connected network, including biases?
2. Why do stacked affine layers without activations remain linear in the input up to a bias?
3. Does universal approximation guarantee successful training?
4. For logits $(0, 0, 0, 0)$, what is the correct-class cross-entropy?
5. What is the ReLU derivative at a strictly negative input?
6. For $z = Wh + b$, why is the weight gradient an outer product?
7. Why clear gradients before a normal independent mini-batch update?
8. With retention probability 0.8, how is retained inverted-dropout activation scaled?
9. Is L2 inside Adam's gradient always equivalent to AdamW decay?
10. Does double descent prove that more capacity always lowers test error?

12.2 Answers

1. $4(3 + 1) + 2(4 + 1) = 26$.
2. Composition multiplies the matrices and combines the biases into one affine map.
3. No; it states that suitable parameters exist under its assumptions.
4. $\log 4 \approx 1.386$ nats.
5. Zero, so this path does not transmit a derivative through that unit.
6. Each weight joins one input component to one output component: gradient is upstream derivative times that input.
7. Frameworks commonly accumulate gradients; otherwise previous batches contribute again unintentionally.

8. Divide by 0.8, multiplying by 1.25 during training.
9. No; adaptive gradient scaling changes the equivalence.
10. No. It is a capacity-dependent phenomenon in some regimes, not a universal guarantee.

13 Source guide

`3.1. Introduction to Deep Learning.pdf`: definitions, applications and representation-learning pipeline. `3.2. Shallow NN.pdf`: ReLU construction, universal approximation, dimensions, activation regions and terminology. `3.3. Deep Learning Fundamentals.pdf`: depth, scores, losses and optimizers. `3.4. Deep Learning Fundamentals II.pdf`: backpropagation, initialization, regularization and generalization. The previously supplied X.1 and X.2 copies yielded identical extracted text to 3.1 and 3.2, so they introduced no additional note scope. All worked numerical networks are explanatory examples of the slide equations.